Highlights

Proxy-guided quality control for tunnel-lining segmentation from point clouds

Xinghui Tao,Zehao Ye,Guangming Wang,Jelena Ninić,Brian Sheil

- Propose a label-free framework for proxy-guided segmentation quality control.
- Calibration uses one labelled subset and one expert reference per lining family.
- Proxy-guided LLM self-refinement outperforms the expert reference across families.
- Control study outperforms random proposals and transfers to an unseen tunnel.
- Refinement gains concentrate where the expert reference produces lower initial accuracy.

Proxy-guided quality control for tunnel-lining segmentation from point clouds

Xinghui Tao[a], Zehao Ye[b], Guangming Wang[a,*], Jelena Ninić[b] and Brian Sheil[a]

[a] *Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, United Kingdom*
[b] *Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, United Kingdom*

ARTICLE INFO

Keywords:
Segmental tunnel lining
Point cloud segmentation
Deployment quality control
Label-free proxy
self-refinement
LLM

ABSTRACT

Point-cloud segmentation of segmental tunnel linings supports automated inspection, but expert settings can degrade across tunnels, and deployment outputs are hard to verify and refine. This paper presents Proxy4Tun, a deployment-label-free framework that estimates segmentation quality from intermediate pipeline metrics and uses the estimate for deployment-time quality control and candidate selection during self-refinement. Across three Seg2Tunnel lining families, uncertainty-guided Bayesian exploration yields 120 calibration runs, from which a four-metric ridge-regression proxy is learned. On 54 held-out runs from 27 held-out subsets (reference/degraded pairs), the proxy achieves Spearman $\rho = 0.789$ and MAE 0.106, and correctly ranks the reference above its degraded counterpart in all pairs. Under a three-attempt budget, three LLM-driven self-refinement improves selected mean mIoU from 0.762 to 0.765–0.787. Control studies shows Proxy4Tun outperforms adaptive-random proposals and transfers to an unseen tunnel. Proxy4Tun is positioned as exploratory QC rather than a certified accuracy verifier.

1. Introduction

Regular inspection of segmental tunnel linings supports structural health assessment and long-term operational safety. However, large tunnel networks and restricted access make manual inspection resource-intensive, motivating cost-effective data collection and analysis [1, 2, 3]. Laser scanning can quickly capture the lining surface as a 3D point cloud [4, 5, 6]. Segmenting the resulting point clouds into individual lining segments then supports automated inspection and predictive-maintenance workflows.

Several point-cloud-based methods exist for tunnel-lining segmentation, but their outputs remain difficult to assess and improve when tunnel geometry or scan quality changes and ground-truth (GT) labels are unavailable. Engineer-designed geometric pipelines rely on hand-crafted rules and thresholds with interpretable checks, yet such changes can still require manual review and parameter re-tuning [4, 6, 7]. Supervised models [8, 9], including foundation-model pipelines [10, 11, 12], can output confidence or uncertainty scores learned during training, but these scores must be calibrated against task-relevant accuracy criteria before they can support deployment-time quality-control (QC) decisions: *accept the output, refine it, or flag it for human review* (Fig. 1).

Broader studies show that segmentation accuracy can be estimated without case-specific GT labels. In medical imaging, for example, quality estimators learn from labelled development data or assess new outputs using labelled reference collections [13, 14, 15]. However, it remains unclear how well these approaches transfer to tunnel point-cloud pipelines, where annotations are limited and parameter settings often rely on domain experts. This motivates our central research question: *Can intermediate metrics from an existing tunnel-segmentation pipeline support accuracy estimation and parameter self-refinement without new GT labels?*

We therefore propose Proxy4Tun, a framework that uses intermediate pipeline metrics as a proxy for segmentation accuracy, enabling deployment-time QC decisions and parameter self-refinement. The proxy is constructed through a controlled calibration study using an adapted SAM4Tun pipeline [16] on three Seg2Tunnel lining families (staggered, continuous, and complex). For each family, calibration uses one labelled subset and one expert-tuned reference parameter setting (three in total). For each reference setting, the pipeline is run to produce a reference output, which is evaluated against GT mIoU.

From these references, we generate 120 calibration runs using bounded random sampling and uncertainty-guided Bayesian exploration. This design spans both high- and low-quality outputs and helps reveal metrics that remain informative across different quality levels. Each run records intermediate pipeline metrics alongside its GT mIoU score, allowing their relationships to be examined.

These metrics fall into two groups: preprocessing quality (PQ) and structural coherency (SC). PQ characterises the completeness and alignment of the depth representation used for segmentation, while SC

*Corresponding author
gw462@cam.ac.uk (G. Wang)

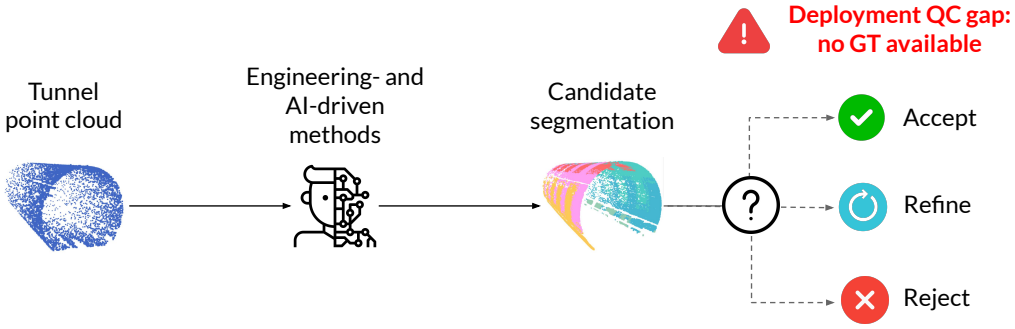


Figure 1: Deployment QC problem: segmentation methods produce candidate outputs, but practitioners still need to decide whether to accept, refine, or flag each result without new GT labels.

measures whether the predicted segmentation is structurally plausible. We compare PQ-only, SC-only, combined, and compact sets, using leave-one-family-out (LOFO) evaluation. Based on these analyses, we freeze a shared four-metric ridge-regression proxy comprising selected metrics from both groups.

The selected proxy is evaluated on 54 held-out runs from 27 subsets (reference/degraded pairs) to test whether it correctly ranks the reference output above its degraded counterpart. We then evaluate whether the proxy can guide LLM-based self-refinement to improve performance beyond the expert-tuned reference. Three LLMs (GPT-5.6, Fable-5.1, and Gemini-3.8) are embeded to assess consistency across models. To assess the contribution of LLM proposals, we compare them with adaptive-random proposals under identical parameter bounds, refinement budgets, and retention rules. We also assess proxy transfer beyond Seg2Tunnel using one unseen STSD tunnel with complex lining layout. In all refinement experiments, the proxy guides selection, while GT mIoU is used only for offline evaluation.

We position Proxy4Tun as an exploratory QC support rather than a accuracy guarantee. Its value lies in using an empirical signal to guide QC decisions and refinement. The unseen tunnel experiment provides initial evidence of transfer beyond Seg2Tunnel, but broader validation across tunnels and lining conditions is still needed.

The contributions of this paper are as follows:

1. A shared PQ/SC accuracy proxy calibrated using one labelled subset and one expert reference per lining family for QC without deployment labels.
2. Correct reference–degradation ranking across all 27 held-out pairs, with initial evidence of proxy transfer to an unseen tunnel with complex ring layouts.
3. Proxy-guided LLM self-refinement that improves held-out mean mIoU over expert-reference settings, with gains concentrated at initial GT mIoU below 0.7.
4. A controlled comparison showing that LLM proposals better preserve accuracy and achieve higher retained mean mIoU than adaptive-random proposals under identical conditions.

The rest of this paper is organised as follows. Section 2 reviews related work. Section 3 presents the methodology and experimental design. Section 4 reports the results, Section 5 discusses findings and limitations, and Section 6 concludes.

2. Related work

The review follows four connected questions: how segmentation methods adapt to changing conditions, how their outputs can be assessed without new GT labels, how calibration data can be obtained with limited annotations, and how assessment feedback can guide self-refinement.

2.1. Tunnel segmentation and adaptation

Methods for tunnel point-cloud segmentation range from geometric pipelines to supervised and foundation-model-based approaches. Engineer-designed pipelines use geometric rules and thresholds to identify lining surfaces and segment boundaries, but their settings may need manual adjustment when tunnel geometry, joint layout, or scan quality changes [4, 6, 7]. PointNet++, RandLA-Net, Mask3D, and TD3D are general point-cloud neural networks that learn geometric representations from labelled 3D datasets [17, 18, 19, 20]. Tunnel-specific supervised methods adapt these ideas with engineering constraints, such as LiningNet [21]. However, labelled tunnel data remain limited, and adapting supervised models to substantially different conditions may require additional annotations and retraining [8, 9].

Foundation-model pipelines reduce the need for task-specific training by using models pre-trained on large datasets. One practical approach is to project point clouds into two-dimensional images that can be processed by vision models. Prompt-based segmentation models include SAM [11], SAM2 [22], and SEEM [23]; infrastructure applications of such models

include crack segmentation [24, 12] and Scan-to-BIM workflows [25, 26, 27].

For tunnel linings, without task-specific model training, SAM4Tun unfolds point clouds into depth images and generates template-based prompts for SAM to segment lining components [16]. However, its preprocessing and prompting rely on many interdependent parameters that assume specific lining geometry and scan conditions. When conditions change, tuning can require substantial effort and domain expertise. R4Tun addresses this parameter-adaptation problem by using a reference configuration, intermediate pipeline metrics, and engineering knowledge to guide LLM-proposed adjustments to SAM4Tun [28]. It adapts without access to GT and uses structured context to propose adjustments. However, it does not provide a mechanism to evaluate the adapted outputs when new GT labels are unavailable, which directly motivates this study.

2.2. Quality estimation without new GT labels

Existing studies estimate segmentation quality from observable properties of the data and segmentation outputs. In medical imaging, Kohlberger et al. use segmentation features to predict overlap error and Dice scores [29], while Robinson et al. learn to predict quality from image–segmentation pairs [14]. For point clouds, LidarMetaSeg combines input features with network confidence measures to estimate the quality of connected semantic segments in self-driving scenes [30]. Consistency between processing outputs can also provide evidence, although agreement does not guarantee correctness: different LiDAR processing streams can agree on an incorrect label [31]. These studies suggest that data properties and output consistency can help assess quality, provided their relationship with actual accuracy is established.

Reference-based approaches provide another source of assessment evidence. Reverse Classification Accuracy (RCA) treats a new segmentation as provisional labels, trains a reverse classifier, and evaluates that classifier on labelled reference images [13]. Its performance on those references is used to estimate the quality of the original segmentation. ConfIC-RCA further uses retrieval of relevant reference images and in-context reverse segmentation to estimate quality [32].

These studies motivate estimating accuracy from measurable intermediate pipeline metrics, avoiding the need for new GT labels at deployment. The question is whether limited labelled tunnel data can support an estimator that remains useful across lining families and tunnels.

2.3. Calibration with limited labelled data

Training a quality estimator requires examples spanning successful and degraded segmentations, but these examples need not require additional scan annotations. Robinson et al. generate multiple segmentations by varying model settings and reuse existing annotations to obtain quality targets [14]. The same labelled input can therefore provide several examples of how processing choices affect accuracy. The challenge is to select useful settings without evaluating every possible combination.

Bayesian optimisation methods motivate incorporating prior knowledge when selecting which settings to evaluate. BOPrO encodes user beliefs about promising parameter regions, while πBO specifies a probability distribution over where the optimum is expected to lie [33, 34]. Whereas these methods seek high-performing settings, this study uses uncertainty-guided Bayesian exploration to collect calibration examples across the learnt parameter space. It starts with a space-filling design across the prescribed ranges [35], fits a Gaussian-process surrogate to the observed accuracy response, and selects additional evaluations where predictive uncertainty is highest to broaden evidence across the parameter space [36, 37]. The same development annotations are reused to calculate mIoU score for each run.

2.4. Feedback-guided self-refinement

LLM-based agents can revise results by proposing changes, evaluating their effects, and using feedback to guide subsequent attempts. Self-Refine asks the model to critique and revise its output, while Reflexion records lessons from task feedback for later attempts [38, 39]. AlphaEvolve and the Darwin Gödel Machine extend feedback-guided search to modifying programs or agent code using automated evaluation [40, 41]. However, repeated revision does not guarantee improvement. Self-correction without verifiable feedback can fail, and increasing an imperfect proxy score can reduce performance when assessed separately [42, 43].

Verifiable feedback can provide diagnostic information to LLM to guide subsequent proposals. ReAct demonstrates how observations from the environment can inform subsequent actions [44]. In our study, PQ and SC metrics provide this feedback context for LLM reasoning: an incomplete depth map or disagreement between detected joints and predicted boundaries can suggest which stages warrant inspection and also expose how each metric contributes to the predicted score, supporting interpretation of the proxy score [45].

3. Methodology

Proxy4Tun has two phases: (i) development-time calibration on labelled scans to learn a proxy from PQ/SC metrics, and (ii) deployment-time scoring and bounded self-refinement on new scans without GT labels. Four requirements shape the design:

i. **Label efficiency.** Calibration should require minimal labelled data and expert input.

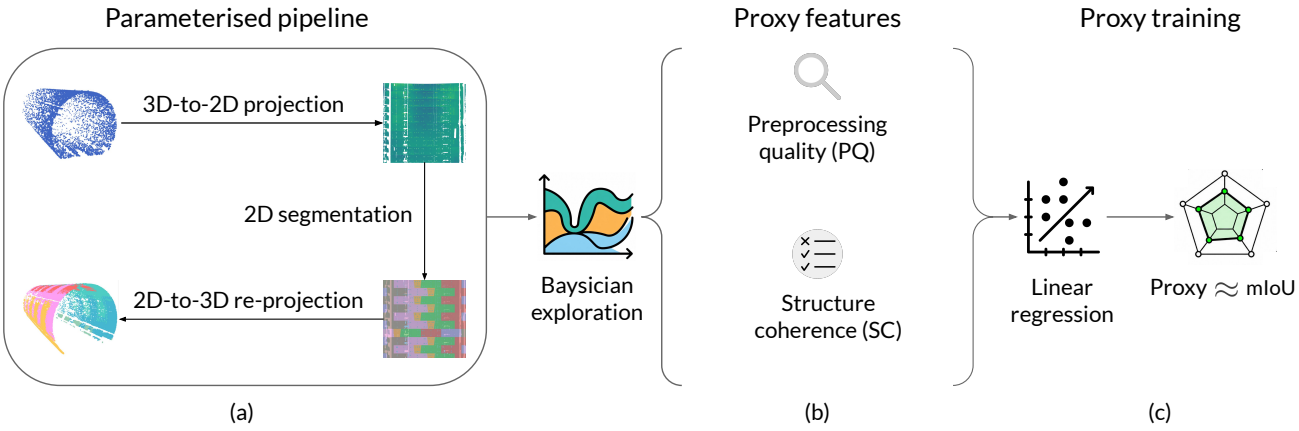


Figure 2: Development and use of the accuracy proxy. Parameter variations generate segmentation outputs with different accuracies. PQ and SC metrics from each run are paired with GT mIoU to train the proxy. Once fitted, the proxy estimates accuracy on new runs using metrics available without GT. The score is an estimate of segmentation accuracy, not a guarantee of correctness.

ii. **Generalisation.** The proxy should remain informative across lining families and transfer to unseen tunnels.
iii. **Auditability.** Each metric's contribution to the proxy score should be interpretable to the LLM proposer.
iv. **LLM value.** LLM proposals should improve refinement beyond random search under comparable conditions.

3.1. Task definition

Given a raw point cloud of a tunnel section, the task is to assign each point either the background label or the label of one lining block. Three block *types* occur in a ring: key (K), adjacent (B), and standard (A). The label set distinguishes the individual blocks within a ring: K, B1, B2, and A1–A3 for staggered and continuous rings (six blocks), and K, B1, B2, and A1–A4 for complex rings (seven blocks), plus background in every family. Seg2Tunnel, the benchmark dataset we use, groups linings into three families (staggered, continuous, and complex). These families differ in ring geometry and acquisition conditions and, most critically for our task, in the key-block (K) layout: ring-to-ring repeating, fixed ring-wise order, and no repeating pattern (Fig. 3).

Given an input point cloud, Proxy4Tun estimates segmentation quality using a learned proxy score computed from PQ/SC metrics. During self-refinement, it evaluates candidate configurations within a predefined parameter space and attempt budget; the proxy score then decides whether to accept the current segmentation, continue refining, or flag the result.

3.2. Framework overview

The framework links a parameterised expert-designed segmetation pipeline, a development-time procedure for learning the mIoU proxy, and a deployment-time self-refinement loop that compares candidates using the learnt proxy (Fig. 2). Throughout, the pipeline algorithms are fixed and only their parameters vary.

3.2.1. Segmentation pipeline

The pipeline is adapted from SAM4Tun [16] and parameterised for the three Seg2Tunnel lining families [46]. Figure 4 illustrates the processing stages.

1. **Unfolding.** RANSAC-based cross-sectional ellipse fitting and centreline estimation establish a cylindrical coordinate system [47]. The lining surface is represented panoramically, with depth describing radial distance from the centreline. Coordinate orientation is fixed so that block positions retain the same meaning across runs.
2. **Denoising.** Radial masks and grid-based point-density filtering remove points inconsistent with the expected lining surface. The filtering follows the density-based principle underlying DBSCAN [48]. Its settings affect both unwanted-point removal and the amount of lining data retained.
3. **Enhancement.** Curvature-guided interpolation and progressive upsampling improve the completeness of the depth representation before detection, while preserving joint boundaries.
4. **Boundary detection.** Orientation-specific Hough transforms identify candidate ring and block boundaries [49]. These detections are organised according to the expected family layout.
5. **Segmentation and reprojection.** The detected layout defines template-based prompts for SAM [11]. The resulting image masks are mapped back to the point cloud to obtain three-dimensional block labels.

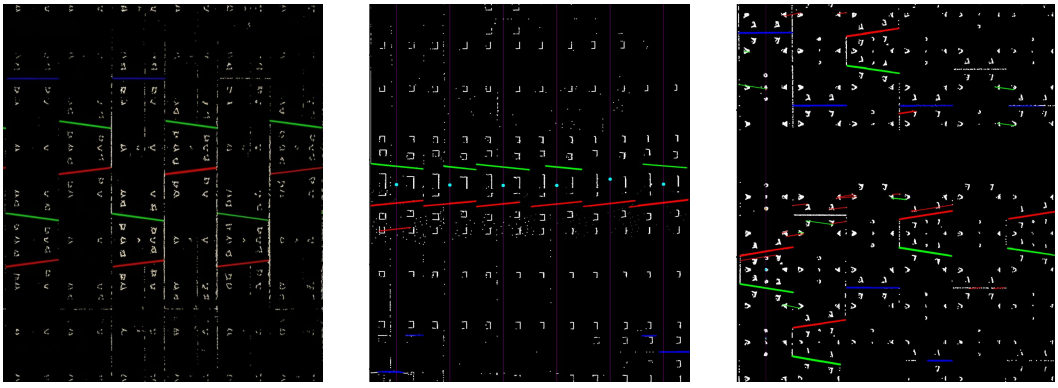


Figure 3: Tunnel-lining block *types* (K, B, A). Evaluation labels distinguish the individual blocks: K, B1, B2, and A1–A3 (six lining classes) or A1–A4 (seven), plus background. mIoU averages over the labels that appear, including background.

This adapted pipeline is used for both calibration and self-refinement. Only its parameter settings vary within prescribed family-specific bounds; the stage algorithms and SAM model weights remain fixed (Appendix A). Each run produces a segmentation output and PQ/SC metrics for proxy-based quality assessment.

3.2.2. Generating calibration examples

Calibration requires examples linking PQ/SC metrics to segmentation accuracy across successful and degraded parameter settings. Each lining family has an expert reference configuration, tuned by the pipeline developers on one labelled development subset before this study. We vary its parameters within prescribed bounds and reuse the same subset's annotations to calculate GT mIoU for each run, producing 120 calibration examples across three development subsets. The family references also provide the starting configurations for self-refinement.

Two sampling stages balance coverage and informativeness within each lining family. First, scrambled Sobol sampling provides 32 valid trials spread across the prescribed parameter ranges [35], allowing exploration of both successful and degraded settings. Second, Bayesian exploration adds eight valid trials where a Gaussian-process (GP) model of GT mIoU has the highest predictive uncertainty [36, 37]. We target uncertainty rather than maximise accuracy because the proxy must estimate and rank segmentation quality across a range of accuracies, rather than identify a single optimum (Appendix B). Each valid run records its parameter configuration, PQ/SC metrics, and GT mIoU. Algorithm 1 and Figure 5 summarise the procedure; Appendix C lists the parameter bounds.

3.2.3. PQ and SC metrics

Metrics form two groups that capture two aspects of pipeline quality: upstream, where the depth representation may be incomplete or misaligned (PQ), and downstream, where the recovered layout may be incoherent even when the input is sound (SC). Table 1 lists the ten candidate metrics (four PQ, six SC), whose relationship with segmentation accuracy is assessed during calibration using GT mIoU. SC is particularly relevant to the complex family, where the key block does not repeat from ring to ring while the retained SC metrics still apply: they quantify fallback assumed lines, match block boundaries to detected joints, and measure how much of the point cloud receives a block label. However, high structural coherence does not guarantee correctness: because joint detections also drive SAM prompting, detected joints and predicted boundaries may agree while sharing the same underlying error. Their agreement is evidence, not proof, of correctness.

3.2.4. Learning the accuracy proxy

The proxy maps PQ and SC metrics to an estimated mIoU. We fit ridge-regression proxies using candidate metric sets and compare them through LOFO validation to select a compact model. Regularisation stabilises estimation under correlated predictors [50, 51], while the linear form allows the estimator to be inspected by the LLMs. The coefficients represent fitted associations, not causal prescriptions for parameter changes. We choose this model form for interpretable feedback and do not claim predictive superiority over nonlinear alternatives.

For a metric vector $\mathbf{x} \in \mathbb{R}^d$, the prediction is

$$\hat{y}(\mathbf{x}) = b + \sum_{j=1}^{d} w_j \frac{x_j - \mu_j}{s_j}, \tag{1}$$

where d is the number of metrics, μ_j and s_j are the training-set mean and standard deviation of metric j, w_j is its fitted coefficient, and b is the intercept. Coefficients are fitted by minimising squared prediction error with an ℓ_2 penalty of strength α.

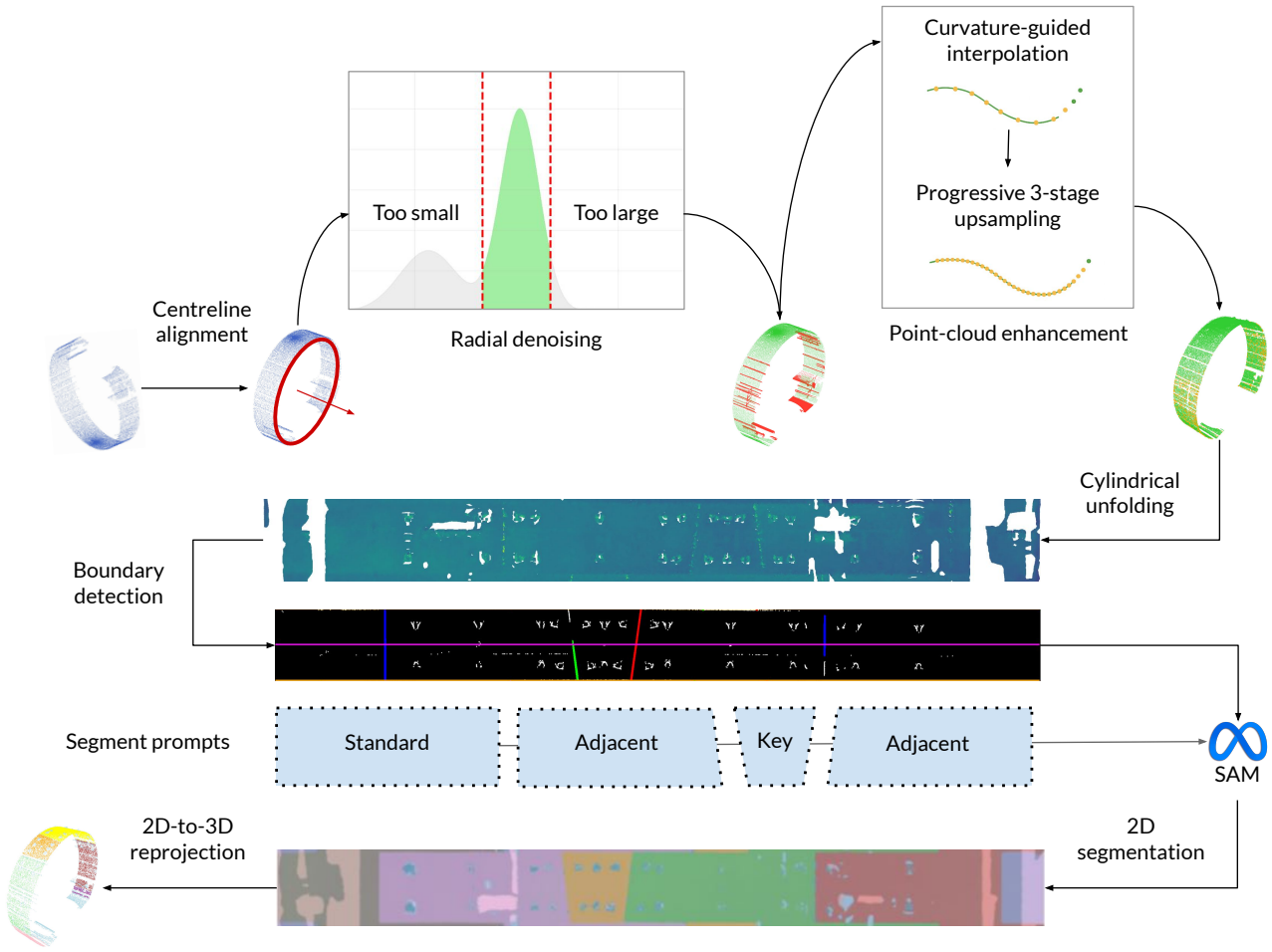


Figure 4: Segmentation pipeline adapted from SAM4Tun. Geometric alignment, filtering, and enhancement produce a depth representation for boundary detection and template-based SAM prompting. The resulting masks are reprojected onto the point cloud. Intermediate outputs provide PQ and SC metrics. The stage algorithms remain fixed; exploration and self-refinement adjust their parameters.

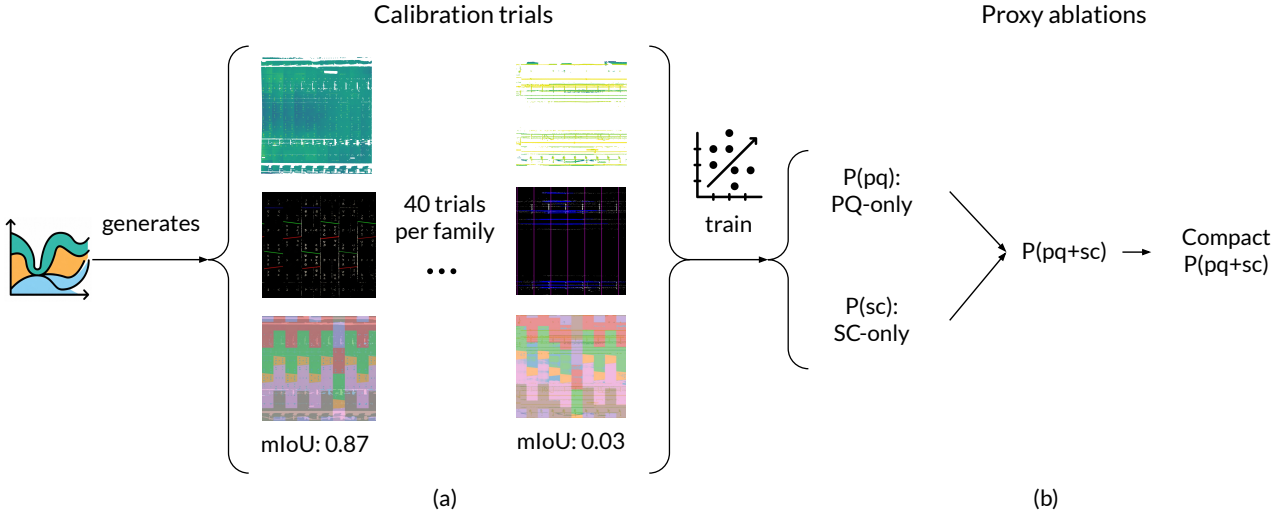


Figure 5: Calibration-example generation and metric comparison. (a) Each lining family contributes 32 valid scrambled-Sobol trials and eight valid Bayesian exploration trials selected by GP uncertainty. All trials reuse the same development-subset annotations. (b) The resulting metrics and GT mIoU values support comparison of PQ-only, SC-only, combined, and compact combined metric sets for the accuracy proxy.

Table 1
Candidate metrics for the mIoU accuracy proxy (four PQ, six SC). Pixel quantities refer to the 5 mm depth-image grid. The four retained by the frozen proxy are marked ⋆. Stages: 1—unfolding; 2—denoising; 3—enhancement; 4—joint detection; 5—SAM segmentation. Combined indices indicate metrics using outputs from both stages.

Group	Measurement	Meaning	Stage
PQ	Unfolding fit residual	Centreline fit error	1
	Orientation agreement	Agreement between the inferred axis and the recorded travel direction	1
	Point retention	Fraction of input points kept after denoising	2
	Missing-depth rate ⋆	Fraction of all depth-image pixels without depth after enhancement	3
SC	Assumed-line count ⋆	Number of fallback joint lines assumed by the detector	4
	Edge support	Fraction of label-boundary pixels within 25 px of a depth edge	3+5
	Correspondence score ⋆	F1 at 15 px between rasterised joint lines and block–block boundaries (empty detection ⇒ 0)	4+5
	Line–boundary distance	Symmetric mean distance (capped at 200 px) between joints and boundaries	4+5
	Fill rate ⋆	Fraction of all input points assigned a non-background label	5
	Phase incoherence	Inconsistency of label ordering across rings	5

Table 2
Prespecified proxy metric sets compared under LOFO validation.

Model	Definition
$P(q)$	Four PQ metrics
$P(c)$	Six SC metrics
$P(q{+}c)$	All ten candidate metrics
Compact $P(q{+}c)$	Four prespecified metrics: one PQ and three SC

We compare four prespecified metric sets: PQ-only $P(q)$, SC-only $P(c)$, combined $P(q{+}c)$, and compact combined models (Table 2). Each LOFO fold trains on two families and evaluates on the third, with metric scaling, penalty selection, and coefficient fitting restricted to the training families. Predictions across the three held-out folds are pooled to calculate Spearman ρ and MAE over all 120 calibration runs; per-family ρ is also reported. The metric set with the highest pooled LOFO ρ is selected, and the final model is fitted on all 120 calibration runs and frozen. Algorithm 2 summarises model development and deployment scoring.

3.3. Proxy-guided parameter self-refinement

The proxy estimates accuracy but does not decide which parameters to change. Each LLM agent (GPT-5.6, Fable-5.1, or Gemini-3.8), evaluated separately, proposes a parameter change (Fig. 6). It receives the family reference, the current run's intermediate outputs and PQ/SC metrics, the proxy score, and short engineering notes linking observable symptoms to likely adjustments (Appendices D and E). It returns one bounded parameter update with a brief rationale.

Shared protocol. Each of the 27 subsets starts from its expert family reference and receives three proposal attempts per arm, without proxy-based admission screening or early stopping. All arms share the parameter bounds, attempt budget, and retention rule. Each attempt modifies only the parameters it calls, and the shared retention rule decides whether that candidate replaces the current output. Invalid or failed proposals consume an attempt, so equal budgets do not necessarily imply equal numbers of pipeline executions or equal runtime.

Adaptive-random control. The control samples the number of parameters to change from an empirical distribution obtained from an earlier LLM pilot, then randomly selects parameters and values within the allowed bounds (Appendix F). This distribution is frozen before the experiment. Random updates are applied to the retained configuration without using intermediate metrics or images to guide sampling. In all arms, however, unfolding parameters are eligible for modification only when the starting centreline residual exceeds 10 cm. The comparison evaluates proposal strategies under common constraints; it does not isolate LLM reasoning from differences in proposal size or parameter choice.

Execution and feedback. Each proposal is checked against the allowed parameter names, types, and ranges. Valid proposals are executed through the pipeline, and the resulting PQ/SC metrics are scored by the frozen proxy. The proposed update, score, and

Algorithm 1 Generating calibration examples through space-filling sampling and Bayesian exploration

Require: Development subset D_f with GT annotations Y_f
Require: Reference configuration P_f^{ref}; expected ring count $N_{\text{exp}} = 10$
Require: Parameter bounds Θ_f; valid-trial budgets $N_0 = 32$, $N_1 = 8$; candidate-pool size $M = 2048$
Ensure: Calibration records $\mathcal{T}_f$

$\mathcal{T}_f \leftarrow \emptyset$; $n_0 \leftarrow 0$; $n_1 \leftarrow 0$
Initialise a scrambled-Sobol sequence
Phase 1: broad initial coverage
while $n_0 < N_0$ **do**
  Draw the next Sobol point and map it to permitted parameter values $\theta \in \Theta_f$
  $P \leftarrow \text{Apply}(P_f^{\text{ref}}, \theta)$
  $R \leftarrow \text{Pipeline}(D_f, P)$
  if R is valid and required metrics are available **then**
    $\mathbf{x} \leftarrow$ Metrics (R, N_{exp})
    $y \leftarrow \text{mIoU}(R, Y_f)$
    Append $(\theta, \mathbf{x}, y)$ to $\mathcal{T}_f$
    $n_0 \leftarrow n_0 + 1$
  else
    Log the failed or incomplete trial
  end if
end while
Phase 2: Bayesian exploration
Fit GP to the (θ, y) pairs in $\mathcal{T}_f$
while $n_1 < N_1$ **do**
  Draw M random candidate settings $\mathcal{P}_t$ within Θ_f
  $\theta \leftarrow \arg\max_{\vartheta \in \mathcal{P}_t} \sigma_t(\vartheta)$
  $P \leftarrow \text{Apply}(P_f^{\text{ref}}, \theta)$
  $R \leftarrow \text{Pipeline}(D_f, P)$
  if R is valid and required metrics are available **then**
    $\mathbf{x} \leftarrow$ Metrics (R, N_{exp})
    $y \leftarrow \text{mIoU}(R, Y_f)$
    Append $(\theta, \mathbf{x}, y)$ to $\mathcal{T}_f$
    Update GP using the expanded $\mathcal{T}_f$
    $n_1 \leftarrow n_1 + 1$
  else
    Log the failed or incomplete trial
  end if
end while
return $\mathcal{T}_f$

outcome are recorded. For the LLM arms, this record provides feedback for subsequent attempts, together with the retained configuration and its outputs as the current state.

Retention. A candidate replaces the currently retained output only if its proxy score, clipped to $[0, 1]$, exceeds the retained score by more than the predefined margin of 0.01; otherwise, the retained output remains unchanged. The same rule applies to all arms. Algorithm 3 summarises the loop.

Algorithm 2 Proxy calibration and deployment scoring

Require: Calibration records $\mathcal{T} = \bigcup_f \mathcal{T}_f$; prespecified candidate metric sets; fixed metric-extraction conventions
Ensure: Frozen proxy g and deployment score $\hat{y}$

for each candidate metric set V **do**
  for each held-out family f **do**
    Use the other two families as training data
    Fit metric scaling and select α using only the training data
    Fit ridge regression and predict the held-out family's mIoU
    Clip predictions to $[0, 1]$
  end for
  Pool predictions across the three folds
  Record pooled Spearman ρ and MAE, and per-family ρ
end for
Select the metric set with the highest pooled LOFO ρ
Fit final scaling and ridge regression on all 120 calibration records
Freeze metric definitions, order, scaler, coefficients, and intercept as g

Receive a completed run r and its expected ring count N_{exp}
$\mathbf{x} \leftarrow$ Metrics (r, N_{exp}) in the frozen metric order
$\hat{y} \leftarrow \min(1, \max(0, g(\mathbf{x})))$
return $\hat{y}$

3.4. Experimental setup

3.4.1. Dataset and reference configurations

We use 30 ten-ring subsets from five parent tunnels in Seg2Tunnel, covering staggered, continuous, and complex lining families [46]. One labelled development subset per family supplies 40 calibration runs; the remaining 27 subsets are held out for evaluation (Table 3). A single proxy is fitted to calibration runs pooled across all three lining families and evaluated on the 27 held-out subsets. Where we report bootstrap confidence intervals, we account for shared geometry and acquisition conditions by resampling at the parent-tunnel level unless stated otherwise (Section 3.4.6).

Each expert family reference was tuned by the pipeline developers on the corresponding development subset, with its annotations available, and established before this study. It provides the basis for calibration

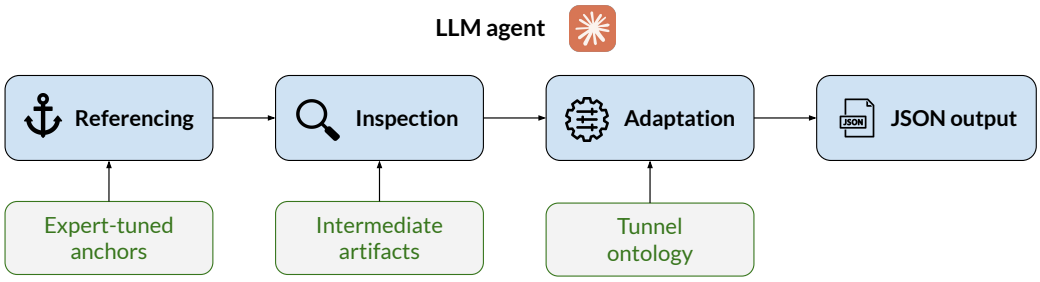


Figure 6: LLM-guided parameter proposals. The agent compares the current run with engineering references, inspects intermediate outputs and PQ/SC metrics, and returns a bounded parameter update with a rationale. After execution, the frozen proxy scores the candidate. Candidate retention depends only on the proxy score.

Algorithm 3 Proxy-guided parameter self-refinement

Require: Input point-cloud subset; expert reference configuration; frozen proxy; parameter bounds
Ensure: Retained configuration, segmentation output, and proxy score

Run the expert reference and retain its configuration and output
Compute its proxy score, clipped to $[0, 1]$
for attempt $= 1, 2, 3$ **do**
    Propose one update using the LLM or adaptive-random strategy
    Validate parameter names, types, bounds, and eligibility
    if the update is invalid **then**
        Record the failure and continue to the next attempt
    end if
    Execute the candidate configuration
    if execution fails **then**
        Record the failure and continue to the next attempt
    end if
    Compute PQ/SC metrics and the candidate proxy score, clipped to $[0, 1]$
    if candidate score $>$ retained score $+ 0.01$ **then**
        Retain the candidate configuration, output, and score
    end if
    Record the update, metrics, score, and retention outcome
    Make the record and retained state available to the next LLM proposal
end for
return retained configuration, segmentation output, and proxy score

and the unchanged starting configuration for held-out evaluation and self-refinement. Development annotations support reference assessment and supply calibration targets; held-out annotations are used only for retrospective evaluation. Appendices A and C list the reference configurations and exploration bounds, respectively.

3.4.2. Metric and model comparisons

We use the 120 calibration runs for two checks. First, we compare how well each metric set fits the training data and predicts accuracy for a family excluded from fitting (LOFO; Table 2). Second, we shuffle GT mIoU values within each family and repeat LOFO fitting 20 times. This breaks run-level relationships while preserving family differences, testing whether the metrics predict accuracy beyond family identity. The final frozen proxy is then evaluated on held-out data.

3.4.3. Paired degradation test

For each of the 27 held-out subsets, we run the family reference and a deliberately degraded configuration (Appendix G). Both configurations process the same input point cloud, avoiding differences between scans. GT mIoU is calculated afterwards to confirm degradation. We evaluate how accurately the frozen proxy estimates mIoU and whether it ranks each reference output above its degraded counterpart. This tests ranking under controlled degradation rather than reliability on naturally occurring failures.

3.4.4. Self-refinement against family-specific references

We compare four proposal arms on all 27 held-out subsets: GPT-5.6, Fable-5.1, Gemini-3.8, and adaptive-random search. Each starts from the same expert family reference and receives three attempts, sharing the frozen proxy, parameter bounds, and retention rule, without admission screening or early stopping (Algorithm 3; Fig. 7). Table 4 lists the study configuration.

Each LLM agent receives the reference, bounds, engineering notes, PQ/SC metrics, proxy scores, and rendered depth maps, detected lines, prompt points, and segmentation images. One session per subset and LLM arm is resumed across attempts to preserve proposal history and feedback, with no access to GT, evaluation directories, or other arms. All 324 proposals and outcomes are logged (27 subsets $\times$4 arms

Table 3
Development and held-out subsets. One labelled development subset per family supplies the calibration trials; nine other subsets per family form the held-out evaluation data.

Property	Staggered (T1, T2)	Continuous (T3)	Complex (T4, T5)
Inner diameter	5.5 m	5.9 m	7.5 m
Ring length	1.2 m	1.2 m	1.8 m
Blocks per ring	6	6	7
Key-block layout	Ring-to-ring repeating pattern	Fixed ring-wise order	No repeating pattern
Scanning	Single-station	Multi-station	Single-station, offset
Development subset	2-1	3-1	5-1
Calibration runs	40	40	40
Held-out subsets	1-1 to 1-5; 2-2 to 2-5	3-2 to 3-10	4-1-to 4-5; 5-2 to 5-5

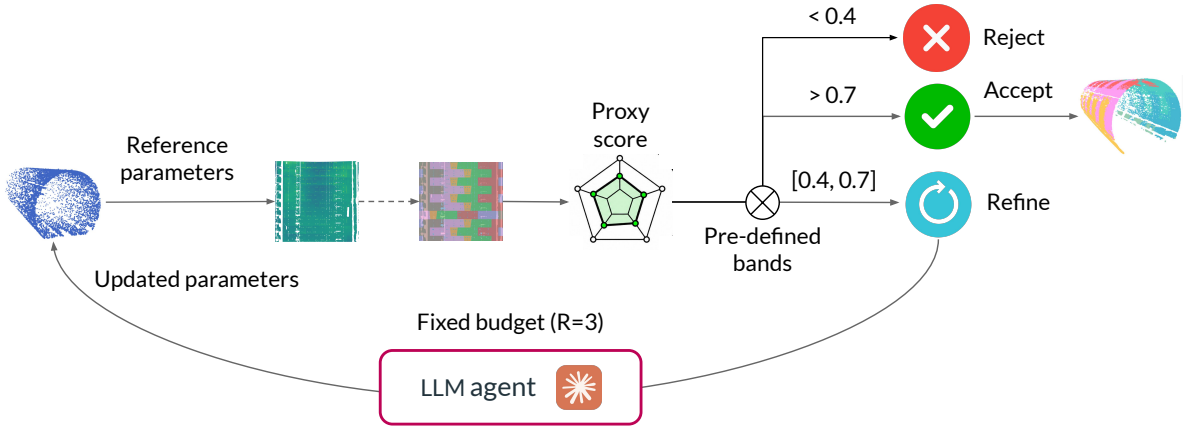


Figure 7: Self-refinement protocol shared by all arms. Each of the 27 expert-reference starts receives three proposal attempts per arm. Invalid or failed attempts consume budget and leave the retained configuration unchanged. A candidate is retained only if its proxy score exceeds the current retained score by more than 0.01.

×3 attempts), comprising 243 LLM proposals and 81 adaptive-random proposals, and pipeline runs use a single NVIDIA RTX 5060 GPU.

3.4.5. Evaluation metrics

Segmentation accuracy is evaluated retrospectively against GT using point-wise semantic mIoU:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \mathrm{mIoU} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{IoU}_c, \tag{2}$$

where TP_c, FP_c, and FN_c are the true-positive, false-positive, and false-negative point counts for label c, and C is the number of labels included in the average for that subset (labels present in either GT or prediction, including background). In this study, $C = 7$ for staggered and continuous subsets and $C = 8$ for complex subsets.

Proxy performance is measured by Spearman ρ for agreement with GT accuracy rankings and mean absolute error (MAE), defined as the mean absolute difference between predicted and GT mIoU. In the paired degradation test, we also report the proportion of pairs in which the reference receives a higher proxy score than its degraded counterpart; ties count as failures.

For self-refinement, we report proxy scores and GT mIoU before and after refinement. For each proposal strategy, we subtract each subset's initial GT mIoU from its final GT mIoU and average these changes across the 27 subsets. Averaging across the 27 subsets gives equal weight to each lining family, since each contributes nine subsets. Proposal quality is assessed relative to the expert-reference start: a proposal is classified as harmful if its GT mIoU is more than 0.05 lower than the starting value.

3.4.6. Statistical uncertainty

The reported mean gain is the unweighted mean of the 27 subset changes. Its interval is a different quantity. Each parent tunnel is first reduced to its own mean gain. A resample draws those five tunnel means with replacement and averages them with equal weight. The 95% interval is the 2.5 and 97.5 percentiles of that average over 100,000 resamples. Equal family counts do not make this the 27-subset mean: the five tunnels contribute 5, 4, 9, 5, and 4 subsets. The error bars in Fig. 9 use this same tunnel-mean interval. Intervals are

Table 4
Self-refinement study configuration.

Setting	Value
Proposal arms	GPT-5.6; Fable-5.1; Gemini-3.8; adaptive-random search
LLM identifiers	`gpt-5.6-sol-medium`; `claude-fable-5-1-thinking-high`; Gemini 3.8 as the active agent model
Interface	One coding-agent session per subset and LLM arm, resumed across attempts
Starting configuration	Expert family reference
Evaluation cases	All 27 held-out subsets
Context (LLM arms)	Engineering notes, pipeline outputs, PQ/SC metrics, and proxy scores; no GT
Budget	Three proposal attempts per subset and arm
Early stopping	None
Output	One JSON parameter update per attempt, with a rationale for LLM arms
Failure handling	Invalid or failed attempts consume budget; retained output remains unchanged
Retention	Candidate proxy score exceeds the current retained score by more than 0.01

interpreted cautiously because only five parent tunnels are available.

3.4.7. Sensitivity to LLM choice

We assess how self-refinement outcomes depend on the proposing LLM by comparing GPT-5.6, Fable-5.1, and Gemini-3.8 under identical starting configurations, parameter bounds, attempt budgets, and proxy-based retention. We compare final mIoU, changes from the expert-reference baseline, harmful-proposal rates, and the numbers of improved, unchanged, and worsened subsets. These comparisons examine whether the observed benefits are consistent across the three models or concentrated in one. Each model is evaluated once per subset, so the analysis describes differences between models rather than variability across repeated sessions.

4. Results

The results follow the three questions of Section 3.4: whether PQ and SC carry accuracy information that transfers across families, whether the frozen proxy separates references from degradations on held-out tunnel sections, and whether it can guide refinement of already-strong references. A residual error analysis then examines failure modes that the four-metric proxy does not reliably detect.

4.1. Proxy calibration and metric ablation

The selected proxy contains three SC metrics (correspondence F1@15, segmentation fill rate, and assumed-line count) and one PQ metric (depth-map invalid-pixel fraction). Assumed-line count is the number of fallback joint lines marked `assume` by the detector. With training-standardised inputs, its fitted equation is

$$\hat{y} = 0.358867 + 0.088443 z_{\text{corr}} + 0.105800 z_{\text{fill}} - 0.021558 z_{\text{assume}} - 0.033997 z_{\text{nan}}. \quad (3)$$

Each z denotes the corresponding metric standardised using the calibration-set mean and standard deviation. RidgeCV selects $\alpha = 5.623413$. Greater correspondence and coverage increase the estimate, while assumed lines and invalid depth decrease it, holding other inputs constant. These are conditional associations in a correlated linear model, not causal effects.

Table 5 and Fig. 8a report the fit. Correlation decreases from 0.863 in training to 0.754 under LOFO. The within-family permutation control gives $\rho = -0.122 \pm 0.093$. Useful ranking therefore remains when a family is excluded and is not explained by family identity alone.

The compact set gives the strongest pooled transfer and lowest MAE (Table 6). The learning curve rises from $\rho = 0.641 \pm 0.174$ at five runs per family to 0.699 ± 0.086 at ten, 0.723 ± 0.062 at 15, 0.753 ± 0.047 at 20, and 0.750 ± 0.032 at 30, reaching 0.754 with all 40. Most transfer performance is therefore present by approximately 20 calibration runs per family, with decreasing variability.

4.2. Discriminating Reference vs. Degraded Outputs

On the 54 held-out runs, the frozen proxy reaches Spearman $\rho = 0.789$ (95% CI $[0.711, 0.858]$) and MAE $= 0.106$ (Table 7; Fig. 8b). It ranks all 27 reference–degradation pairs correctly. The highest degraded score is 0.179 and the lowest reference score is 0.412, so a 0.4 threshold, chosen retrospectively within this gap, flags all 27 degraded outputs and none of the references. These constructed pairs demonstrate discrimination; they do not independently validate the threshold.

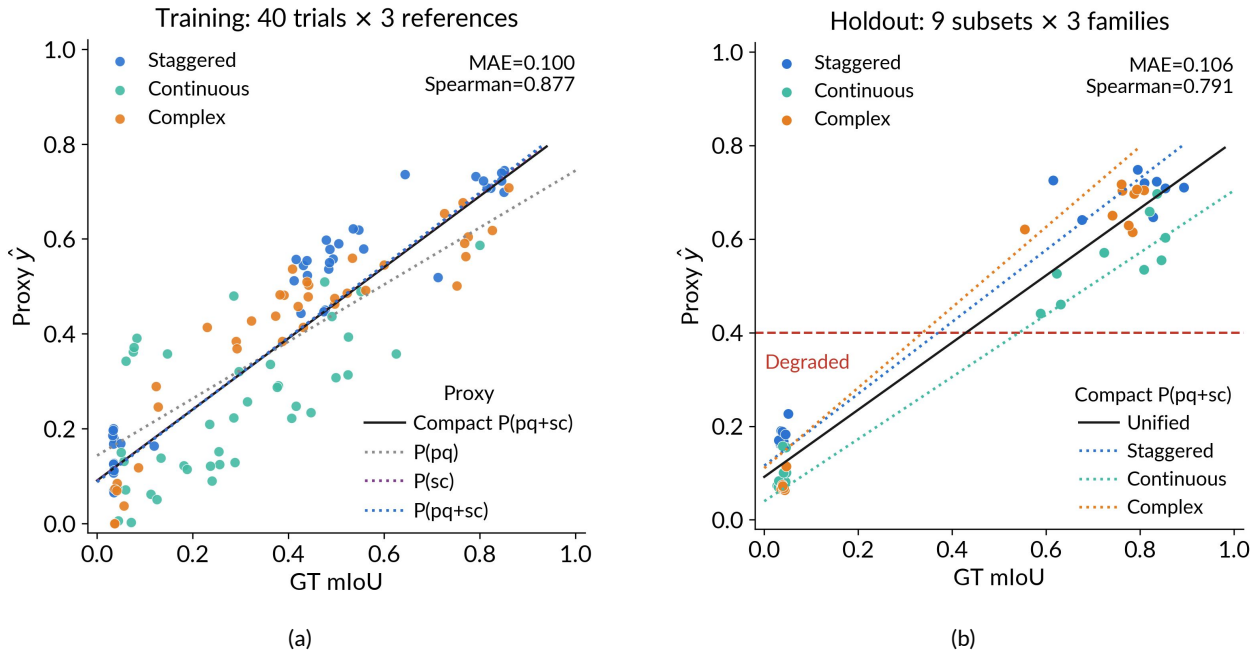


Figure 8: Proxy training and holdout discrimination. (a) The frozen four-metric Ridge proxy on 120 calibration trials (40 per family). (b) The same proxy, without refitting, on 54 held-out runs: 27 family-specific reference outputs and 27 constructed degraded counterparts. Lines connect each reference–degraded pair.

Table 5
Proxy calibration and LOFO transfer for the compact model.

Setting	MAE	Spearman
Training fit ($n = 120$)	0.108	0.863
Permutation control (20 shuffles, LOFO)	–	-0.122 ± 0.093
Leave-one-family-out	0.145	0.754

The same proxy remains informative on 324 naturally generated proposal outputs: $\rho = 0.516$ (95% CI $[0.400, 0.680]$) and MAE $= 0.141$. All 18 outputs with GT mIoU below 0.3 score below 0.4; the highest score among them is 0.294. At a proxy threshold of 0.4, recall is 1.000, 0.958, and 0.839 for GT mIoU below 0.3, 0.4, and 0.5, respectively. Among the 27 expert-reference starts, the operating bands assign one to accept, 26 to refine, and none to flag.

Table 6
Proxy metric-set comparison under LOFO. Family columns give Spearman correlation on each held-out family.

Condition	#Metrics	Pooled ρ	Staggered	Continuous	Complex	LOFO MAE
$P(q)$ (PQ only)	4	0.615	0.595	0.551	0.466	0.166
$P(c)$ (SC only)	6	0.715	0.834	0.358	0.866	0.155
$P(q{+}c)$ (combined)	10	0.744	0.869	0.263	**0.936**	0.148
Compact $P(q{+}c)$ (deployed)	**4**	**0.754**	**0.883**	**0.479**	0.861	**0.145**

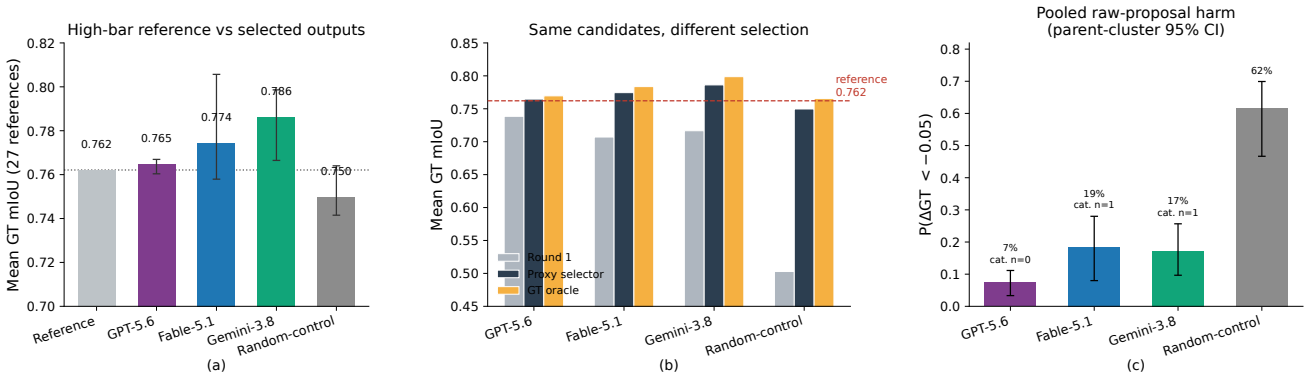


Figure 9: Proxy-guided self-refinement against family-specific references. (a) Selected mean GT mIoU for the shared reference start and four proposal arms on all 27 held-out cases (error bars: the tunnel-mean 95% interval in Table 8, drawn on the paired gain). (b) Counterfactual selection policies on the same candidates. (c) Harmful-proposal rate: share of proposals whose GT mIoU is more than 0.05 lower than the start ($\Delta\text{mIoU} < -0.05$; annotation: population std of proposal Δ).

Table 7
Holdout fidelity of the four-metric proxy (54 runs: 27 family-specific references and 27 constructed degraded overlays).

Metric	Value
Ranking agreement	
Spearman ρ	0.789
Bootstrap 95% CI (subset; pairs together)	[0.711, 0.858]
Prediction error	
MAE (overall)	0.106
Paired ranking and retrospective flagging	
Pair ranking (reference > degraded)	27/27
Retrospective threshold $\hat{y} < 0.4$: precision / recall	1.000 / 1.000

4.3. Self-refinement from expert-reference configurations

The expert-reference configurations start at mean GT mIoU 0.762. Proxy-guided self-refinement reaches 0.765 for GPT-5.6, 0.774 for Fable-5.1, and 0.787 for Gemini-3.8, whereas adaptive-random reaches 0.754 (Table 8; Fig. 9a). All three LLM arms therefore finish above the common start in mean accuracy, while random search finishes below it. Individual selections can still deteriorate. Before selection, the harmful-proposal rates are 0.07, 0.19, and 0.17 for GPT-5.6, Fable-5.1, and Gemini-3.8, compared with 0.62 for adaptive-random (Fig. 9c).

The aggregate means conceal where improvement occurs (Table 9). Fable-5.1 and Gemini-3.8 gain 0.078 and 0.143 on the six starts below 0.7, but change by −0.006 and −0.009 on the 21 stronger starts. GPT-5.6 is nearly flat in both groups, and random search is slightly negative overall. This concentration is retrospective; starting GT mIoU is not used for admission or selection.

4.4. External STSD control

On the additional STSD subset, nine controlled proposals are evaluated from a fixed intermediate state with proxy score 0.5578, zero assumed lines, and first-10 mIoU 0.7277. Eight proposals are rejected and one angular adjustment is retained at proxy score 0.5664, zero assumed lines, and first-10 mIoU 0.7276. This is initial evidence from one external control without an STSD-specific expert reference, not a second panel benchmark; the complete audit is given in Appendix I.

4.5. Error analysis: block-identity shifts

Case 1-5 illustrates a blind spot. Its start has proxy score 0.610. Fable-5.1 reaches score 0.636 with GT mIoU 0.884, and Gemini-3.8 reaches score 0.644 with mIoU 0.888: large accuracy changes produce only modest proxy movement (Fig. 10b).

The K-block position anchors block identities around each ring. Incorrect K-block localisation can shift the label sequence while preserving geometric boundaries, fill, depth completeness, assumed-line count, and joint–boundary agreement. The proxy can therefore miss an important identity correction or select a geometrically coherent but mislabeled output. The largest losses in Table 8 show the converse risk. More reliable K-block localisation or independent block-identity evidence is needed; a coherence metric derived from the same layout may remain insensitive to the error.

5. Discussion

The results answer the research question of Section 1 in two parts. Intermediate metrics do carry accuracy information that survives transfer to held-out tunnel sections and, less evenly, to unseen families. Proxy-based retention reduces losses relative to accepting proposals unconditionally, but selection errors can

Table 8
Self-refinement on all 27 held-out family-specific references. Means give equal weight to all subsets and, consequently, to the three lining families. Selected means use the shared proxy selector with a retention margin: a candidate replaces the retained output only if its proxy score exceeds the retained score by more than 0.01. The 95% CI is for the equal-weight mean of the five parent-tunnel gains, not for the 27-subset mean in Mean Δ. Improved/unchanged/worsened counts use GT mIoU changes above 0.005, within ±0.005, and below −0.005, respectively.

Arm	Selected mean	Mean Δ	95% CI for mean gain	Up/same/down	Worst Δ
Reference start	0.762	–	–	–	–
GPT-5.6	0.765	+0.003	[−0.002, 0.005]	3/22/2	−0.013
Fable-5.1	0.774	+0.012	[−0.004, 0.042]	4/20/3	−0.143
Gemini-3.8	0.787	+0.025	[0.005, 0.037]	7/14/6	−0.185
Random search	0.754	−0.008	[−0.016, 0.002]	3/21/3	−0.129

Table 9
Concentration of selected gains by starting GT mIoU. The split at 0.7 is retrospective and is not an admission rule.

Arm	Mean Δ (start < 0.7)	Mean Δ (start ≥ 0.7)	Spearman(start,Δ)	Leave-one-parent range
Fable-5.1	+0.078	−0.006	−0.252	0.004–0.026
Gemini-3.8	+0.143	−0.009	−0.406	0.017–0.029
GPT-5.6	+0.010	+0.001	−0.258	0.001–0.004
Random search	+0.001	−0.011	−0.252	−0.010–−0.005

still reduce true accuracy. Final performance depends on both proposal quality and proxy accuracy.

5.1. Key findings

Fewer metrics transfer better. The design principle of Section 3 was to select for transfer, not fit, and the ablation bears this out: adding six metrics to the compact four lowered LOFO ρ from 0.754 to 0.744. With 120 runs from three development subsets, a proxy that explains its own calibration set is not the goal; one that still orders runs on a family it has not seen is. The cost is coverage: the four retained metrics do not detect a block-identity shift.

Retention limits losses and can still select a worse output. Proxy-based retention reduces losses relative to accepting proposals unconditionally, but selection errors can still reduce true accuracy. Final performance depends on both proposal quality and proxy accuracy. Adaptive-random retention reaches 0.754 from the 0.762 start. The same rule retains the losses in Table 8. The control compares proposal strategies under the same bounds, attempt budget, and

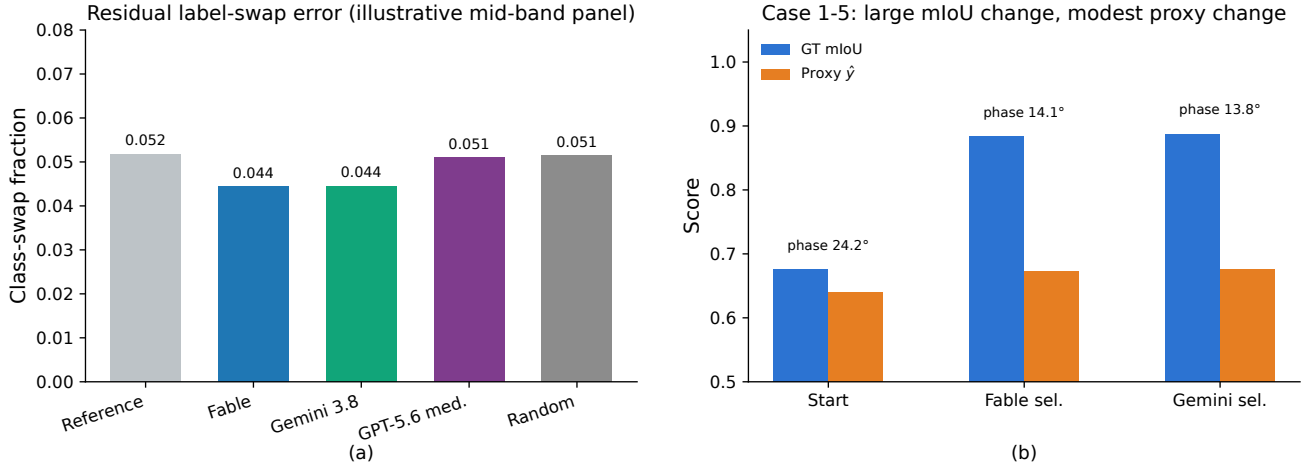


Figure 10: Block-identity shifts that the proxy does not reliably detect. (a) Point-level error composition for the reference start and each arm's selected outputs. Percentages are over all points in the evaluated point cloud: *missed* = GT non-background but predicted background; *spurious* = GT background but predicted non-background; *swapped* = GT non-background but predicted as the wrong non-background class. (b) Case 1-5: large GT mIoU gains and phase-incoherence (an angular measure, in degrees, of label-order inconsistency across rings) reductions accompany only modest movement of the four-metric proxy score.

Table 10
Positioning Proxy4Tun against representative tunnel-segmentation paradigms. The comparison concerns setup cost and deployment capability as well as accuracy. mIoU figures are *not* measured under comparable settings: supervised and expert-tuned figures are taken from prior work under their original protocols, whereas Proxy4Tun figures are from this study (27 held-out family-specific references).

Method	Deep learning training	Per-case tuning	Deployment-label-free quality signal	mIoU (setting)
LiningNet/SparseUNet/SCF-Net/FA-ResNet	✓	–	–	0.83–0.90
SAM4Tun	–	Expert	–	above 0.90
Proxy4Tun (four-metric + LLM)	–	**Auto**	✓	**0.765–0.787**

retention rule; it does not isolate the use of pipeline evidence from differences in proposal size or parameter choice (Section 3.3).

What deployment would look like. Each pipeline release still needs one expert reference per family and one calibration pass. From then on the frozen proxy supplies a label-free score that can be monitored over time and used as the selection signal inside the bounded self-refinement loop; three bounded attempts cost roughly 15 GPU-minutes. On this study's constructed reference–degraded pairs, a proxy threshold of 0.4 (chosen retrospectively within the observed separation gap) flags all degradations and none of the references, but this does not by itself validate a deployment cutoff. Self-refinement is assisted, auditable reconfiguration against a strong engineering baseline, not autonomous inspection.

5.2. Comparison with state-of-the-art

Comparisons here concern operating points, not accuracy leaderboards (Table 10). Supervised 3D models on Seg2Tunnel report strong in-distribution accuracy [46] but require per-point labels and provide no deployment-time accept/refine/flag gate. Expert-tuned SAM4Tun reaches high accuracy on curated rings [16], but a fixed configuration emits no signal when it degrades. Relative to that static pipeline, Proxy4Tun adds an explicit quality signal and a bounded self-refinement loop. The only directly comparable contrast in this study is LLM-plus-proxy versus random-plus-proxy under the shared reference starts and selection rule. The mIoU gap to expert-tuned SAM4Tun in Table 10 is therefore not a like-for-like deficit: those figures come from curated rings with per-case expert tuning, whereas the Proxy4Tun figures are 27 held-out subsets with three automated attempts each.

5.3. Limitations

Four limitations bound the claims. First, the evidence base is narrow: 120 calibration runs come from three development subsets. Some held-out subsets come from the same parent tunnels; tunnels 1 and 4 are other tunnels in this collection. Transfer is shown to held-out tunnel sections within this collection, not to new projects or lining geometries, and the weak continuous-family LOFO result ($\rho = 0.479$) shows that even within-collection transfer is uneven. The single STSD subset is an external control, not a general external benchmark. Second, the four metrics do not detect a block-identity shift from incorrect K-block localisation, so a higher proxy score can accompany lower GT accuracy, and the two largest LLM losses passed the selection rule for this reason. Third, the degradations are constructed and the panel is small: the natural proposals arise from the same bounded refinement experiment rather than an independent deployment stream, and with five parent tunnels the bootstrap intervals for GPT-5.6 and Fable-5.1 include zero, so their gains are descriptive. Fourth, the loop is one expert reference and three attempts; nothing here shows sustained improvement over longer horizons or without a reference to start from.

6. Conclusions

This paper asked whether intermediate tunnel-segmentation metrics can estimate accuracy and guide parameter self-refinement on held-out tunnel sections without new GT labels. Proxy4Tun answers with a four-metric Ridge proxy calibrated from 120 runs around one expert reference per lining family, selected for cross-family transfer rather than fit, and frozen before any held-out scan is scored.

- The metrics carry transferable accuracy information. Three structure-coherence metrics and one preprocessing-quality metric reach Spearman 0.789 on 54 held-out runs and order every reference above its degraded partner; adding more metrics hurt transfer.
- From a 0.762 baseline, LLM proposals under the selection rule reach 0.765–0.787, while adaptive-random proposals under the same rule reach 0.754. Proxy-based retention reduces losses relative to accepting proposals unconditionally, but selection errors can still reduce true accuracy.
- Final performance depends on both proposal quality and proxy accuracy.

The principal remaining limit is a block-identity shift: the four metrics can approve an output whose K-block localisation is wrong. Future work should add reliable K-block localisation or independent block-identity evidence, test the loop on naturally degraded runs and on tunnels outside this collection, and examine whether the proxy can guide refinement without an expert reference to start from. Until then, Proxy4Tun is a label-free assessment and assisted-reconfiguration tool, not an autonomous inspector.

Data availability

The source code, parameter snapshots, exploration-trial tables, and evaluation scripts will be released upon publication. The Seg2Tunnel dataset is publicly available [46].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI use

During this study, LLM agents (GPT-5.6, Fable-5.1, and Gemini-3.8) were evaluated as proposal mechanisms in the self-refinement loop, together with a random-search control under the same proxy-based selection rule. LLMs were also used for language editing and for improving manuscript structure. The authors reviewed and edited all content and take full responsibility for the content of the publication.

References

[1] L. Attard, C. J. Debono, G. Valentino, M. D. Castro, Tunnel inspection using photogrammetric techniques and image processing: A review, ISPRS Journal of Photogrammetry and Remote Sensing 144 (2018) 180–188.

[2] R. Montero, J. G. Victores, S. MartÃηnez, A. JardÃşn, C. Balaguer, Past, present and future of robotic tunnel inspection, Automation in Construction 59 (2015) 99–112.

[3] A. Strauss, J. Bien, H. Neuner, C. Harmening, C. Seywald, M. Österreicher, et al., Sensing and monitoring in tunnels: testing and monitoring methods for the assessment of tunnels, Structural Concrete 21 (2020) 1356–1376.

[4] M. Q. Huang, J. Ninić, Q. B. Zhang, BIM, machine learning and computer vision techniques in underground construction: current status and future perspectives, Tunnelling and Underground Space Technology 108 (2021) 103677.

[5] A. SjÃűlander, V. Belloni, A. Ansell, E. NordstrÃűm, Towards automated inspections of tunnels: a review of optical inspections and autonomous assessment of concrete tunnel linings, Sensors 23 (2023) 3189.

[6] L. Weidner, G. Walton, Generalized extraction of bolts, mesh, and rock in tunnel point clouds: a critical comparison of geometric feature-based methods using random forest and neural networks, Remote Sensing 16 (2024) 4466.

[7] Z. Wang, Z. Zhu, Y. Wu, Q. Hong, D. Jiang, J. Fu, et al., Automated tunnel point cloud segmentation and extraction method, Applied Sciences 15 (2025) 2926.

[8] C. Su, Q. Hu, Z. Yang, R. Huo, A review of deep learning applications in tunneling and underground engineering in china, Applied Sciences 14 (2024) 1720.

[9] Y.-J. Cha, R. Ali, J. Lewis, O. Büyüköztürk, Deep learning-based structural health monitoring, Automation in Construction 161 (2024) 105328.

[10] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, et al., On the opportunities and risks of Foundation Models, 2021. URL: https://arxiv.org/abs/2108.07258. doi:10.48550/arXiv.2108.07258. arXiv:2108.07258.

[11] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, et al., Segment anything, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 4015–4026. doi:10.1109/ICCV51070.2023.00371.

[12] Z. Ye, L. Lovell, A. Faramarzi, J. Ninić, Sam-based instance segmentation models for the automation of structural damage detection, Advanced Engineering Informatics 62 (2024) 102826.

[13] V. V. Valindria, I. Lavdas, W. Bai, K. Kamnitsas, E. O. Aboagye, A. G. Rockall, D. Rueckert, B. Glocker, Reverse classification accuracy: Predicting segmentation performance in the absence of ground truth, IEEE Transactions on Medical Imaging 36 (2017) 1597–1606.

[14] R. Robinson, O. Oktay, W. Bai, V. V. Valindria, M. M. Sanghvi, N. Aung, J. M. Paiva, F. Zemrak, K. Fung, E. Lukaschuk, A. Lee, V. Carapella, Y. J. Kim, B. Kainz, S. K. Piechnik, S. Neubauer, S. Petersen, C. Page, D. Rueckert, B. Glocker, Real-time prediction of segmentation quality, in: Medical Image Computing and Computer Assisted Intervention – MICCAI 2018, volume 11073 of *Lecture Notes in Computer Science*, 2018, pp. 578–585. doi:10.1007/978-3-030-00937-3_66.

[15] J. Fournel, A. Bartoli, D. Bendahan, M. Guye, et al., Medical image segmentation automatic quality control: A multi-dimensional approach, Medical Image Analysis 74 (2021) 102213.

[16] Z. Ye, W. Lin, A. Faramarzi, X. Xie, J. Ninić, SAM4Tun: no-training model for tunnel lining point cloud component segmentation, Tunnelling and Underground Space Technology 158 (2025) 106401.

[17] C. R. Qi, L. Yi, H. Su, L. J. Guibas, PointNet++: deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017, pp. 5105–5114. URL: https://proceedings.neurips.cc/paper/7095-pointnet-deep-hierarchical-feature-learning-on-point-sets-in-a-metric-space.

[18] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, et al., RandLA-Net: efficient semantic segmentation of large-scale point clouds, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 11105–11114. doi:10.1109/CVPR42600.2020.01112.

[19] J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, B. Leibe, Mask3D: mask transformer for 3D semantic instance segmentation, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2023, pp. 8216–8223. doi:10.1109/ICRA48891.2023.10160590.

[20] M. Kolodiazhnyi, A. Vorontsova, A. Konushin, D. Rukhovich, Top-down beats bottom-up in 3D instance segmentation, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024, pp. 3566–3574. doi:10.1109/WACV57701.2024.00353.

[21] W. Lin, B. Sheil, P. Zhang, K. Li, X. Xie, Structural geometry-informed 3D deep learning for segmental tunnel lining analysis in point clouds, Automation in Construction

176 (2025) 106281.

[22] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, C. Feichtenhofer, SAM 2: Segment anything in images and videos, 2024. URL: https://arxiv.org/abs/2408.00714. arXiv:2408.00714.

[23] X. Zou, J. Yang, H. Zhang, F. Li, L. Li, J. Wang, L. Wang, J. Gao, Y. J. Lee, Segment everything everywhere all at once, 2023. URL: https://arxiv.org/abs/2304.06718. arXiv:2304.06718, neurIPS 2023 (preprint).

[24] K. Ge, C. Wang, Y. Guo, Y. Tang, Z. Hu, H. Chen, Fine-tuning vision foundation model for crack segmentation in civil infrastructures, Construction and Building Materials 431 (2024) 136573.

[25] B. Wang, Z. Chen, M. Li, Q. Wang, C. Yin, J. C. Cheng, Omni-scan2bim: A ready-to-use scan2bim approach based on vision foundation models for MEP scenes, Automation in Construction 163 (2024) 105384.

[26] F. Pan, S. Jeon, B. Wang, F. McKenna, S. X. Yu, Zero-shot building attribute extraction from large-scale vision and language models, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024, pp. 8647–8656. URL: https://openaccess.thecvf.com/content/WACV2024/html/Pan_Zero-Shot_Building_Attribute_Extraction_From_Large-Scale_Vision_and_Language_Models_WACV_2024_paper.html.

[27] Z. Ye, Q. Li, G. Desiderio, M. Huang, W. Liu, V. Villa, J. Ninić, Maintenance-oriented tunnel digital model generation via panoptic segmentation of ultra-high-resolution images, Computer-Aided Civil and Infrastructure Engineering (2026) 100032.

[28] X. Tao, Z. Ye, G. Wang, J. Ninić, B. Sheil, R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds, 2026. Manuscript.

[29] T. Kohlberger, V. Singh, C. Alvino, C. Bahlmann, L. Grady, Evaluating segmentation error without ground truth, in: Medical Image Computing and Computer-Assisted Intervention – MICCAI 2012, volume 7510 of *Lecture Notes in Computer Science*, Springer, 2012, pp. 528–536. URL: https://doi.org/10.1007/978-3-642-33415-3_65. doi:10.1007/978-3-642-33415-3_65.

[30] P. Colling, M. Rottmann, L. Roese-Koerner, H. Gottschalk, False positive detection and prediction quality estimation for LiDAR point cloud segmentation, in: 2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI), IEEE, 2021, pp. 18–25. URL: https://arxiv.org/abs/2110.15681. doi:10.1109/ICTAI52525.2021.00011.

[31] D. Bogdoll, F. Sartoris, V. Geppert, S. Pavlitska, J. M. Zöllner, Label-free model failure detection for lidar-based point cloud segmentation, in: 2025 IEEE Intelligent Vehicles Symposium (IV), IEEE, 2025, pp. 1–8. URL: https://arxiv.org/abs/2407.14306. doi:10.1109/IV64158.2025.11163519.

[32] M. Cosarinsky, R. Billot, L. Mansilla, G. Jimenez, N. Gaggion, G. Fu, T. Tirer, E. Ferrante, ConfIC-RCA: Statistically grounded efficient estimation of segmentation quality, IEEE Transactions on Medical Imaging 45 (2026) 3545–3557.

[33] A. Souza, L. Nardi, L. B. Oliveira, K. Olukotun, M. Lindauer, F. Hutter, Bayesian optimization with a prior for the optimum, arXiv preprint arXiv:2006.14608 (2020).

[34] C. Hvarfner, D. Stoll, A. Souza, M. Lindauer, F. Hutter, L. Nardi, πBO: Augmenting acquisition functions with user beliefs for Bayesian optimization, in: International Conference on Learning Representations, 2022. URL: https://arxiv.org/abs/2204.11051.

[35] S. Joe, F. Y. Kuo, Constructing Sobol sequences with better two-dimensional projections, SIAM Journal on Scientific Computing 30 (2008) 2635–2654.

[36] R. B. Gramacy, H. K. H. Lee, Adaptive design and analysis of supercomputer experiments, Technometrics 51 (2009) 130–145.

[37] R. B. Gramacy, Surrogates: Gaussian Process Modeling, Design and Optimization for the Applied Sciences, Chapman & Hall/CRC, Boca Raton, Florida, 2020. URL: https://bookdown.org/rbg/surrogates/.

[38] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, Self-refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems, volume 36, 2023, pp. 46534–46594. URL: https://arxiv.org/abs/2303.17651.

[39] N. Shinn, B. Labash, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, 2023. URL: https://arxiv.org/abs/2303.11366. doi:10.48550/arXiv.2303.11366. arXiv:2303.11366.

[40] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, M. Balog, AlphaEvolve: A coding agent for scientific and algorithmic discovery, arXiv preprint arXiv:2506.13131 (2025).

[41] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune, Darwin Gödel Machine: Open-ended evolution of self-improving agents, arXiv preprint arXiv:2505.22954 (2025).

[42] Firstname. Huang, et al., On the limits of intrinsic self-correction in large language models, 2024. Preprint. (Please replace this placeholder with the full bibliographic metadata.).

[43] L. Gao, J. Schulman, J. Hilton, Scaling laws for reward model overoptimization, in: Proceedings of the 40th International Conference on Machine Learning, volume 202 of *Proceedings of Machine Learning Research*, PMLR, 2023, pp. 10835–10866. URL: https://proceedings.mlr.press/v202/gao23h.html.

[44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, 2023. URL: https://arxiv.org/abs/2210.03629. doi:10.48550/arXiv.2210.03629. arXiv:2210.03629, international Conference on Learning Representations (ICLR).

[45] C. Rudin, Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead, Nature Machine Intelligence 1 (2019) 206–215.

[46] W. Lin, B. Sheil, P. Zhang, B. Zhou, C. Wang, X. Xie, Seg2Tunnel: a hierarchical point cloud dataset and benchmarks for segmentation of segmental tunnel linings, Tunnelling and Underground Space Technology 147 (2024) 105735.

[47] M. A. Fischler, R. C. Bolles, Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography, Communications of the ACM 24 (1981) 381–395.

[48] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), AAAI Press, 1996, pp. 226–231.

[49] R. O. Duda, P. E. Hart, Use of the Hough transformation to detect lines and curves in pictures, Communications of the ACM 15 (1972) 11–15.

[50] A. E. Hoerl, R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, Technometrics 12 (1970) 55–67.

[51] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss,

Table 11
Pipeline anchor parameters: expert-tuned, family-specific anchor parameter profiles of the segmentation pipeline.

Stage	Parameter	Description	Staggered / Continuous / Complex
Unfolding	`diameter`	Cylindrical scale.	5.5 / 5.9 / 7.5 m
Unfolding	`slice_spacing_factor`	Slice spacing (ring width).	1.2 / 1.2 / 1.8 m
Unfolding	`polynomial_degree`	Centreline fit degree.	3 / 2 / 2
Unfolding	`residual_recentre`	Per-bin frame recentring.	off / on / on
Denoising	`mask_r_low`	Inner radial gate.	2.33 / 2.85 / 3.65 m
Denoising	`mask_r_high`	Outer radial gate.	2.78 / 3.0 / 3.9 m
Denoising	`y_step`	Angular bin width.	0.4 / 0.5 / 0.1 m
Denoising	`z_step`	Radial histogram bin width.	0.005 / 0.001 / 0.001 m
Denoising	`grad_threshold`	Outer-edge gradient threshold.	0.15 / 0.2 / 0.2
Denoising	`smoothing_window_size`	Cutoff-curve smoothing window.	5 / 3 / 3
Denoising	`smoothing_offset`	Post-smoothing cutoff bias.	−0.002 / −0.003 / −0.005 m
Denoising	`default_cutoff_z`	Fallback radial cutoff.	2.75 / 2.85 / 3.65 m
Enhancing	`upsampling_stage1`	Stage-1 upsampling spacing.	0.06 / 0.08 / 0.09 m
Enhancing	`upsampling_stage2`	Stage-2 upsampling spacing.	0.03 / 0.04 / 0.045 m
Enhancing	`upsampling_stage3`	Stage-3 upsampling spacing.	0.015 / 0.02 / 0.0225 m
Enhancing	`curvature_threshold`	Panel–joint curvature threshold.	0.005 / 0.0005 / 0.0005
Enhancing	`depth_threshold_low`	Lower interpolation depth tolerance.	0.005 / 0.01 / 0.0065 m
Enhancing	`depth_threshold_high`	Upper interpolation depth tolerance.	0.015 / 0.01 / 0.013 m
Enhancing	`inter_radius`	Interpolation neighbour radius.	0.03 / 0.06 / 0.06 m
Enhancing	`n_segment_start/end`	Interpolated ring-index window.	0–5 / 2–8 / 5–14
Enhancing	`ring_spacing_factor`	Nominal ring width.	1.2 / 1.2 / 1.8 m
Detecting	`binary_threshold`	Depth-map binarisation level.	125 / 127 / 125
Detecting	`hough_thresh_obliq`	Oblique joint-line vote threshold.	65 / 30 / 35
Detecting	`hough_thresh_horiz`	Horizontal joint-line vote threshold.	65 / 30 / 35
Detecting	`hough_thresh_vert`	Ring-boundary vote threshold.	600 / 1500 / 5000
Detecting	`minLineLength_obliq`	Minimum oblique line length.	100 / 40 / 80 px
Detecting	`minLineLength_horiz`	Minimum horizontal line length.	100 / 50 / 80 px
Detecting	`maxLineGap_obliq`	Oblique line-merging gap limit.	45 / 30 / 60 px
Detecting	`maxLineGap_horiz`	Horizontal line-merging gap limit.	10 / 10 / 15 px
Detecting	`angle_range_oblique`	Oblique-joint angle band.	±[6, 9] / ±[4, 10] / ±[5, 10]°
Segmenting	`segment_per_ring`	Segments per lining ring.	6 / 6 / 7
Segmenting	`segment_order`	Circumferential segment order.	K,B1,A1–A3,B2 / K,B2,A3–A1,B1 / K,B1,A1–A4,B2
Segmenting	`segment_width`	Nominal segment width.	1200 / 1200 / 1800 mm
Segmenting	`K_height`	Key-segment arc height.	1079.9 / 823.8 / 1227.0 mm
Segmenting	`AB_height`	Standard-segment arc height.	3239.8 / 3346.8 / 3726.9 mm
Segmenting	`angle`	Oblique K-joint inclination.	7.52 / 6.12 / 9.8°
Segmenting	`processing.y_bounds`	Valid panorama band.	3800–13300 / 4500–14000 / 4200–13100 px

V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, Scikit-learn: Machine learning in python, Journal of Machine Learning Research 12 (2011) 2825–2830.

A. Pipeline anchor parameters

This appendix summarises the expert-tuned *anchor* parameter profiles used to warm-start the segmentation pipeline for each lining family (staggered, continuous, complex). Table 11 lists the fixed stage algorithms and the corresponding family-specific parameter values that define the reference configurations used throughout the study.

B. Bayesian exploration configuration

Table 12 records the frozen design-time exploration settings used to generate the 120 calibration trials. Accepted trials are exactly N_0=32 Sobol + N_{GP}=8 GP-uncertainty acquisitions per family (40 valid trials each).

C. Per-family search-space bounds

This appendix reports the full bounded per-family search spaces B used for Bayesian exploration. Table 13 enumerates every optimised parameter and its allowable range (or discrete set) for each family reference

Table 12
Design-time Bayesian exploration configuration (frozen snapshot).

Setting	Value
Target accepted trials / family	N=40 (N_0=32 Sobol, N_{GP}=8 accepted GP)
Sobol	Scrambled QMC in the unit cube
GP kernel	Matérn ν=2.5 + white noise; `normalize_y=True`
Length-scale bounds	$(10^{-2}, 10^{2})$ per dimension; noise $(10^{-6}, 10^{-1})$
GP optimiser	`GaussianProcessRegressor`, $n_{\mathrm{restarts_optimizer}}$=3
Acquisition	Maximise predictive std over random candidate pool $\mathcal{P}_{\mathrm{BO}}$ of size 2048
Failure / top-up rule	Status not ok or required metrics incomplete ⇒ exclude; continue until accepted targets met

case, defining the constraints enforced during the frozen 120-trial campaign.

D. Tunnel ontology structure

The reflective agent consults a staged SAM4Tun ontology that encodes pipeline artifacts, per-stage parameters and quality metrics, failure modes, diagnostic rules, remediation actions, and inter-stage dependencies. The top-level schema is summarised below: We omit the complete YAML and show only the structure plus one representative stage example.

Listing 1: Ontology top-level keys and stage inventory.

```
meta # version, domain, coordinate systems
artifacts # data objects (cloud, depth map, masks, ...)
stages[] # pipeline stages (unfolding through segmentation):
  stage_1_centreline
  stage_2_denoise
  stage_3_upsample
  stage_4_detection
  stage_5_segmentation
inter_stage_dependencies[] # upstream cause -> defer local retune
diagnostic_rules[] # metric patterns -> failure modes
remediation_actions[] # bounded parameter changes
agent_interface # {state, diagnosis, proposal}
```

Listing 2: Example ontology stage: radial denoising failure modes.

```
- id: stage_2_denoise
  name: Local density-difference-based denoising
  quality_metrics:
    - id: m_retained_ratio
      healthy: "lining kept; background removed"
    - id: m_bg_leakage
      healthy: "near zero"
  failure_modes:
    - id: f_over_gate
      name: Radial gate too tight
      symptom: "lining removed; holes in later depth map"
    - id: f_under_gate
      name: Radial gate too loose
      symptom: "rails/tube/noise retained downstream"
```

E. Agent prompt structure

Each reflection round issues a single bounded JSON overlay. The model-facing prompt is assembled from a fixed skeleton (role, isolation rules, current state, GT-blind intrinsics, four-metric proxy scores, allowed artifact images, family parameter bounds, and the required output schema). Listing 3 summarises the skeleton; Listing 4 gives one example proposal. GT mIoU and evaluation directories are never included.

Listing 3: Agent-prompt skeleton (per round).

```
Role: reflective parameter agent for SAM4Tun
Allowed context:
  - family reference priors / warm-start parameters
  - GT-blind intrinsics and four-metric proxy scores
  - stage artifacts (depth map, detections, SAM masks)
  - ontology + bounded family search space
State: subset, family, reference proxy, operating band
Task: diagnose failure mode; propose ONE overlay within bounds
Output JSON keys (required):
  observation, failure_mode, rationale, overlay
```

Listing 4: Example JSON proposal returned by the agent.

```
{
  "observation": "Centreline residual is high; unroll shows
       asymmetric voids.",
  "failure_mode": "f_curve_underfit (under-fit centre curve)",
  "rationale": "Raise polynomial degree and tighten RANSAC before
       retuning SAM.",
  "overlay": {
    "unfolding": {
      "polynomial_degree": 3,
      "ransac_threshold": 0.9
    }
  }
}
```

F. Random-search control

The random-search control samples sparse overlays. Let k be the number of parameters changed in a round. We draw k from a frozen empirical distribution estimated from an earlier LLM pilot campaign (198 proposals across three LLM arms), then sample k distinct parameters uniformly (without replacement) from the bounded search space and sample each chosen parameter value uniformly within its family-specific bounds. That pilot distribution does not match the present LLM proposals: this experiment records a mean of 8.64 changed parameters for random against 4.11–7.35 for the LLM arms. Unfolding parameters are

eligible only when the start's centreline residual exceeds 10 cm, matching the constraint applied to LLM arms. Parameter identities and values are otherwise random; the overlay is applied to the retained configuration, and retention uses the proxy score. The distribution and seed were frozen before any LLM attempt in this study, so the control cannot track the LLM runs it is compared with.

Frozen sampling design. Seed base 20260915; the generator for case index i and round r is seeded with $20260915 + 1000\,i + r$. Change-count probabilities $\Pr(k)$, $k = 1, \ldots, 15$:

k	1	2	3	4	5	6	7	8
$\Pr(k)$	.036	.005	.025	.030	.046	.061	.066	.051
k	9	10	11	12	13	14	15	
$\Pr(k)$	.076	.071	.157	.188	.107	.076	.005	

A draw of k larger than the number of eligible parameters is clipped to that number.

G. Predefined parameter perturbations

Each held-out subset is paired with one family-specific overlay taken from the lowest-GT-mIoU valid calibration trial of that family's development subset. Unfolding keys are dropped so the perturbation does not change the centreline; the remaining stage-2 to stage-5 keys are applied to the family reference. The three overlays were frozen before any held-out scoring. All 54 runs completed, so no perturbation was replaced.

Staggered (source 2-1/s017, source GT mIoU 0.032). `mask_r_low`= 2.207, `mask_r_high`= 2.741, `z_step`= 0.00633, `grad_threshold`= 0.183, `curvature_threshold`= 7.14×10^{-4}, `inter_radius`= 0.0353, `binary_threshold`= 125, `hough_threshold_oblique`= 46, `hough_threshold_horizontal`= 60, `hough_threshold_vertical`= 520, `maxLineGap_oblique`= 36, `processing.y_bounds`= [4003, 13100].

Continuous (source 3-1/s030, source GT mIoU 0.042). `mask_r_low`= 2.756, `mask_r_high`= 2.906, `mask_theta_low`= 2.063, `mask_theta_high`= 18.79, `curvature_threshold`= 1.55×10^{-3}, `binary_threshold`= 106, `hough_threshold_oblique`= 32, `hough_threshold_horizontal`= 37, `hough_threshold_vertical`= 2457, `pattern_tolerance`= 13, `segment_width`= 1315, `K_height`= 944, `angle`= 5.56°.

Complex (source 5-1/g001, source GT mIoU 0.036). `mask_r_low`= 3.776, `mask_r_high`= 3.961, `curvature_threshold`= 2.42×10^{-4}, `binary_threshold`= 159, `hough_threshold_oblique`= 33, `hough_threshold_horizontal`= 26, `hough_threshold_vertical`= 4596, `maxLineGap_oblique`= 40, `maxLineGap_horizontal`= 22, `pattern_tolerance`= 15, `segment_width`= 1847, `K_height`= 1014, `angle`= 8.35°.

H. Assumed-line proxy parameters

Table 14 gives the frozen scaler and coefficients used for the assumed-line result analysis. The intercept is 0.3588667 and the ridge penalty is $\alpha = 5.623413$.

I. STSD external-control audit

The fixed starting context has proxy score 0.5578, zero assumed lines, and first-10 mIoU 0.7277. The nine controlled follow-up proposals are listed in Table 15.

J. Full-panel selection audit

Table 16 summarises all 27 held-out family-specific references and the selected offline mIoU under each proposal arm. Starts are the family-specific reference configurations. Selected values follow the shared proxy selector with a retention margin: a candidate replaces the retained output only if its proxy score exceeds the retained score by more than 0.01.

Table 13
Complete per-family search-space bounds B for the frozen 120-trial campaign.

Family (case)	Parameter	Bounds
Staggered (2-1)	unfolding.ransac_threshold	$[0.8, 1.2]$
	unfolding.slice_spacing_factor	$[1.05, 1.35]$
	unfolding.polynomial_degree	$\{2, 3\}$
	unfolding.random_seed	$\{0, \ldots, 7\}$
	denoising.mask_r_low	$[2.2, 2.5]$
	denoising.mask_r_high	$[2.7, 2.9]$
	denoising.z_step	$[0.001, 0.008]$
	denoising.grad_threshold	$[0.1, 0.25]$
	enhancing.curvature_threshold	$[3\times10^{-4}, 8\times10^{-3}]$
	enhancing.inter_radius	$[0.02, 0.08]$
	detecting.binary_threshold	$[100, 160]$
	detecting.hough_threshold_oblique	$[40, 90]$
	detecting.hough_threshold_horizontal	$[40, 90]$
	detecting.hough_threshold_vertical	$[400, 800]$
	detecting.maxLineGap_oblique	$[20, 80]$
	sam.y_bounds_lower	$[3600, 4400]$
Continuous (3-1)	unfolding.ransac_threshold	$[0.8, 1.2]$
	unfolding.slice_spacing_factor	$[1.05, 1.35]$
	unfolding.polynomial_degree	$\{2, 3\}$
	unfolding.random_seed	$\{0, \ldots, 7\}$
	denoising.mask_r_low	$[2.7, 3.0]$
	denoising.mask_r_high	$[2.9, 3.15]$
	denoising.mask_theta_low	$[1.0, 2.5]$
	denoising.mask_theta_high	$[15, 19]$
	enhancing.curvature_threshold	$[2\times10^{-4}, 2\times10^{-3}]$
	detecting.binary_threshold	$[100, 160]$
	detecting.hough_threshold_oblique	$[20, 60]$
	detecting.hough_threshold_horizontal	$[20, 60]$
	detecting.hough_threshold_vertical	$[800, 2500]$
	detecting.maxLineGap_oblique	$[10, 60]$
	detecting.pattern_tolerance	$[5, 20]$
	detecting.uniform_k_snap	$\{\text{off}, \text{on}\}$
	sam.segment_width	$[1000, 1400]$
	sam.K_height	$[700, 1000]$
	sam.angle	$[4, 9]$
Complex (5-1)	unfolding.ransac_threshold	$[0.8, 1.2]$
	unfolding.slice_spacing_factor	$[1.6, 2.0]$
	unfolding.polynomial_degree	$\{2, 3\}$
	unfolding.random_seed	$\{0, \ldots, 7\}$
	denoising.mask_r_low	$[3.5, 3.8]$
	denoising.mask_r_high	$[3.75, 4.05]$
	enhancing.curvature_threshold	$[2\times10^{-4}, 2\times10^{-3}]$
	detecting.binary_threshold	$[100, 160]$
	detecting.hough_threshold_oblique	$[25, 70]$
	detecting.hough_threshold_horizontal	$[25, 70]$
	detecting.hough_threshold_vertical	$[800, 5000]$
	detecting.maxLineGap_oblique	$[30, 80]$
	detecting.maxLineGap_horizontal	$[5, 30]$
	detecting.pattern_tolerance	$[5, 20]$
	sam.segment_width	$[1500, 2100]$
	sam.K_height	$[1000, 1500]$
	sam.angle	$[7, 12]$

Table 14
Frozen assumed-line proxy parameters.

Metric	Calibration mean	Calibration SD	Coefficient
Correspondence F1@15	0.1652637	0.1517009	$+0.0884432$
SAM fill rate	0.4568573	0.2836379	$+0.1058000$
Assumed-line count	4.5916667	3.6594531	-0.0215581
Missing-depth rate	0.4074803	0.3033190	-0.0339974

Table 15
STSD control from the fixed intermediate state.

Arm	Round	Proposed update	Outcome	Proxy	Assumed lines	mIoU_{10}
GPT-5.6	1	`prompt_logic=notebook_12`	Rejected	0.5578	0	0.7277
GPT-5.6	2	`AB_height=3220`	Rejected	0.3601	0	0.7147
GPT-5.6	3	`y_bounds=[3000,12900]`	Rejected	0.5326	0	0.7285
Gemini-3.8	1	`prompt_logic=t12_pattern`	Rejected	0.5218	0	0.6018
Gemini-3.8	2	`y_bounds=[2800,13100]`	Rejected	0.5311	0	0.7292
Gemini-3.8	3	`key_pair_max_sep_px=300`	Rejected	0.5578	0	0.7277
Fable-5.1	1	`padding=250`	Rejected	0.5280	0	0.7293
Fable-5.1	2	`y_bounds=[2800,13100]`	Rejected	0.5311	0	0.7292
Fable-5.1	3	`angle=8.0`	Retained	0.5664	0	0.7276

Table 16
Merged 27-case panel: reference start and selected GT mIoU by arm.

Subset	Family	Start	GPT-5.6	Fable-5.1	Gemini-3.8	Random
1-1	staggered	0.784	0.784	0.784	0.783	0.784
1-2	staggered	0.809	0.809	0.809	0.809	0.809
1-3	staggered	0.827	0.827	0.827	0.827	0.827
1-4	staggered	0.853	0.855	0.853	0.851	0.853
1-5	staggered	0.676	0.676	0.884	0.888	0.676
2-2	staggered	0.615	0.615	0.877	0.880	0.615
2-3	staggered	0.795	0.790	0.793	0.803	0.795
2-4	staggered	0.835	0.835	0.807	0.835	0.835
2-5	staggered	0.893	0.880	0.893	0.708	0.893
3-2	continuous	0.631	0.634	0.634	0.754	0.643
3-3	continuous	0.808	0.808	0.808	0.812	0.808
3-4	continuous	0.622	0.622	0.619	0.622	0.605
3-5	continuous	0.588	0.642	0.588	0.847	0.598
3-6	continuous	0.836	0.837	0.693	0.821	0.836
3-7	continuous	0.845	0.845	0.845	0.845	0.786
3-8	continuous	0.723	0.723	0.723	0.711	0.630
3-9	continuous	0.820	0.820	0.820	0.820	0.820
3-10	continuous	0.853	0.853	0.854	0.854	0.724
4-1	complex	0.741	0.742	0.741	0.741	0.741
4-2	complex	0.787	0.787	0.775	0.781	0.787
4-3	complex	0.554	0.554	0.554	0.554	0.554
4-4	complex	0.784	0.808	0.807	0.807	0.669
4-5	complex	0.808	0.808	0.808	0.808	0.808
5-2	complex	0.793	0.793	0.793	0.784	0.793
5-3	complex	0.762	0.762	0.761	0.762	0.762
5-4	complex	0.760	0.760	0.760	0.754	0.760
5-5	complex	0.775	0.780	0.801	0.780	0.794
Mean	–	0.762	0.765	0.774	0.787	0.754